HỌC VIỆN / KHOA CÔNG NGHỆ THÔNG TIN

HỌC PHẦN: LẬP TRÌNH BACKEND CHUYÊN SÂU

BÁO CÁO KẾT QUẢ THỰC HÀNH

CHƯƠNG 08: TRUY VẤN NÂNG CAO & PHÂN TRANG - XỬ LÝ LỖI TẬP TRUNG

Sequelize ORM · SQL Server · Express 5 Async Error Flow · React Vite

Thông Tin Đề Án & Tác Giả	
Họ và tên sinh viên:	**ĐẶNG BẮC NAM**
Mã dự án (Repository):	**BACKEND-20261001-DANGBACNAM**
Công nghệ áp dụng:	Node.js (v24), Express 5, TypeScript (ESM), Sequelize 6, SQL Server, Vite, React 19, Axios
Phạm vi triển khai:	Lab 1: Phân trang + Search tại DB; Lab 2: Global Error Middleware; Mở rộng: Stored Procedure, Raw Query, JOIN

1. Định Hướng, Mục Tiêu & Khảo Sát Đầu Vào (Slide 01, 04, 05)

Chương 08 phát triển trực tiếp nối tiếp từ dự án backend-quan-ly-sinh-vien đã hoàn thành ở Chương 07 (bao gồm xác thực JWT, phân quyền RBAC và upload ảnh đại diện đại diện Multer). Toàn bộ cơ sở dữ liệu SQL Server, các bảng dbo.SinhVien, dbo.Users, dbo.Lop, các middleware xác thực và giao diện React Vite được giữ nguyên cấu trúc.

Mục tiêu kỹ thuật của bài thực hành gồm các nội dung cốt lõi:

- **Phân trang & Tìm kiếm tại Database (Lab 1):** Thực hiện phân trang trực tiếp tại SQL Server bằng Sequelize findAndCountAll với offset và limit. Sắp xếp ổn định ORDER BY maSV ASC. Kết hợp lọc theo họ tên (hoTen) bằng toán tử Op.like trước khi đếm tổng bản ghi và chia trang. Không dùng Array.slice() ở phía client.
- **Xử lý Lỗi Toàn Cục & Express 5 (Lab 2):** Sử dụng cơ chế chuyển lỗi tự động của Express 5; chuẩn hóa cấu trúc phản hồi lỗi { success: false, status, message, data: null }. Đảm bảo che giấu thông tin nhạy cảm (SQL text, secrets, stack trace) khi chạy môi trường production.

Ba quy tắc bất biến trong kiến trúc:

- **pnpm + TypeScript + ESM/NodeNext:** Giữ nguyên cấu trúc module ESM với phần mở rộng .js khi import nội bộ.
- **Cơ sở dữ liệu SQL Server:** Không dùng sync({ force: true }) hay sync({ alter: true }); không drop dữ liệu thực tế.
- **Bảo toàn Auth & Upload:** Giữ nguyên middleware isAuth, isAdmin và chức năng upload avatar của Chương 06 và Chương 07.

2. Hợp Đồng API & Hai Luồng Nghiệp Vụ Xuyên Suốt (Slide 02, 03)

Hệ thống thực hiện phân định rõ hai luồng điều phối request xuyên suốt ứng dụng:

Luồng A — Xem trang danh sách có phân trang & tìm kiếm:

Vite/React (chọn trang 2, từ khóa "Anh") → Axios gửi HTTP GET /api/v1/sinh-vien?page=2&limit=2&search=Anh → Express Route gọi parseSinhVienQuery validate query string → Controller gọi SinhVienService.layDanhSachSinhVien → Sequelize thực thi findAndCountAll() sinh câu lệnh SQL Server: WHERE hoTen LIKE N'%Anh%' ORDER BY maSV ASC OFFSET 2 ROWS FETCH NEXT 2 ROWS ONLY → Server trả JSON đúng hợp đồng: data là mảng + metadata pagination → React nhận dữ liệu và cập nhật bảng.

Luồng B — Xử lý lỗi tập trung:

Client gửi tham số sai (?page=-1 hoặc ?limit=abc) → parseSinhVienQuery phát hiện sai quy chuẩn, throw new AppError(400, "...") → Express 5 tự động chuyển lỗi tới Global Error Middleware đặt cuối chuỗi pipeline → Handler bắt lỗi, ghi log nội bộ và trả về HTTP 400 với data: null. Khi tìm sinh viên không tồn tại (/SV999), Service phát sinh AppError(404, "Không tìm thấy sinh viên") để Global Handler xuất JSON 404 nhất quán.

Tham số Query	Quy ước kỹ thuật	Giá trị mặc định & Ví dụ
page	Số nguyên dương >= 1; nếu sai trả 400 Bad Request	Mặc định: 1 (Ví dụ: ?page=2)
limit	Số nguyên trong khoảng 1 đến 50; nếu sai trả 400 Bad Request	Mặc định: 10 (Ví dụ: ?limit=5)

search	Chuỗi tìm kiếm theo hoTen; tự động trim khoảng trắng; tối đa 80 ký tự	Mặc định: "" (Ví dụ: ?search=Anh)
offset	Backend tự tính toán theo công thức: (page - 1) * limit	page=2, limit=5 -> offset=5
order	Sắp xếp cố định ORDER BY maSV ASC đảm bảo tính ổn định giữa các trang	Luôn luôn maSV ASC

Cấu trúc phản hồi thành công thống nhất: Trường data là một mảng thuần túy (ISinhVienDTO[]), metadata pagination được đặt ở cấp cao nhất cùng với success và message. Thiết kế này bảo đảm client cũ truy cập response.data.data hoạt động bình thường mà không bị gián đoạn.

3. Lab 1 — Phân Trang & Tìm Kiếm Tại SQL Server (Slide 06, 07, 08, 09, 10, 11)

3.1. Công thức phân trang & Bài toán tính nhanh:

Công thức tính offset: **`offset = (page - 1) * limit`** Bài tính nhanh trên lớp: Nếu page = 5 và limit = 20 thì offset được tính như sau: **offset = (5 - 1) * 20 = 4 * 20 = 80.** Quy ước trường hợp không có dữ liệu (totalItems = 0): Hệ thống trả về data: [], totalPages: 0; giá trị page vẫn giữ nguyên số trang mà client yêu cầu.

3.2. Truy vấn T-SQL tương đương trong SQL Server:

```
SELECT [maSV], [hoTen], [email], [maLop] FROM [dbo].[SinhVien] AS [SinhVien] WHERE [SinhVien].[hoTen] LIKE N'%Anh%' ORDER BY [SinhVien].[maSV] ASC OFFSET 2 ROWS FETCH NEXT 2 ROWS ONLY;
```

- ORDER BY maSV ASC: Tạo thứ tự ổn định giữa các trang; bắt buộc phải có khi dùng mệnh đề OFFSET/FETCH. **- OFFSET 2:** Bỏ qua đúng 2 bản ghi đầu tiên của tập kết quả. **- FETCH NEXT 2:** Chỉ đọc tối đa 2 bản ghi kế tiếp của trang hiện tại. **- COUNT(*):** Đếm tổng số bản ghi thỏa điều kiện lọc để tính tổng số trang: totalPages = Math.ceil(totalItems / limit).

3.3. Giải thích câu hỏi tự suy ngẫm (Slide 11):

"Một sinh viên Lan Anh đang ở vị trí thứ 30 theo maSV. Vì sao cách lấy 10 dòng đầu rồi mới filter sẽ không tìm thấy bạn ấy?" Trả lời: Nếu lấy 10 dòng đầu theo mã sinh viên (từ dòng 1 đến dòng 10) nạp vào bộ nhớ rồi mới dùng JavaScript filter theo họ tên, thì sinh viên Lan Anh ở vị trí thứ 30 hoàn toàn không nằm trong 10 dòng được nạp về, dẫn đến kết quả trả về mảng rỗng. Vì vậy, điều kiện lọc (WHERE hoTen LIKE ...) bắt buộc phải được thực thi tại Database trước khi thực hiện COUNT(*) và OFFSET/FETCH.

3.4. Triển khai Validation Query String (src/validations/sinhVien.query.ts):

Hàm parseSinhVienQuery kiểm tra: page phải là số nguyên dương >= 1; limit phải là số nguyên dương từ 1 đến 50; search cắt khoảng trắng thừa bằng trim() và giới hạn không quá 80 ký tự. Mọi trường hợp sai đều phát sinh AppError(400, message).

3.5. Triển khai Service Sequelize (src/services/sinhVien.service.ts):

Sử dụng SinhVien.findAndCountAll({ where, limit: query.limit, offset: (query.page - 1) * query.limit, order: [["maSV", "ASC"]] }). Trả về đối tượng gồm mảng data và metadata pagination { page, limit, totalItems, totalPages }.

4. Lab 1 (Tùy Chọn) — Gắn Phân Trang Vào Vite + React + Axios (Slide 12)

Giao diện React Vite (frontend/src/components/SinhVienTable.tsx) được tích hợp các thành phần tương tác phục vụ phân trang và tìm kiếm:

- **Ô tìm kiếm theo họ tên:** Gắn trực tiếp với state search. Khi người dùng thay đổi từ khóa, hệ thống tự động thiết lập lại page = 1 để tránh rơi vào trang trống.
- **Nút điều hướng Trang trước / Trang sau:** Nút Trước bị vô hiệu hóa (disabled) khi đang ở trang 1; nút Sau bị vô hiệu hóa khi đang ở trang cuối cùng (page >= totalPages).
- **Hộp chọn số lượng hiển thị (Limit Selector):** Hỗ trợ các tùy chọn 2, 5, 10, 20 sinh viên/trang. Khi thay đổi limit, trang được đặt lại về 1.
- **Thông tin tiến trình:** Hiển thị nhãn: "Trang X / Y (Tổng số: Z sinh viên)" được đồng bộ trực tiếp từ metadata pagination của backend.

5. Lab 2 — Xử Lý Lỗi Toàn Cục & Bảo Mật Express 5 (Slide 15, 16, 17)

Cơ chế xử lý lỗi của Express 5 tự động chuyển tiếp Promise Rejection phát sinh trong async route handler mà không cần bọc asyncHandler hay try/catch thủ công tại từng controller. Error Middleware 4 tham số (err, req, res, next) đảm nhiệm:

- **Lớp lỗi AppError:** Kế thừa từ class Error gốc, bổ sung statusCode và cờ isOperational = true để phân biệt lỗi nghiệp vụ có chủ đích với lỗi lập trình.
- **Kiểm tra res.headersSent:** Nếu response header đã được gửi tới client từ trước, lập tức ủy quyền cho next(err) để ngăn lỗi crash kết nối.
- **Chuẩn hóa định dạng phản hồi:** Mọi lỗi đều xuất ra JSON thống nhất: { success: false, status: number, message: string, data: null }.
- **Bảo vệ bí mật máy chủ (Lỗi 500):** Ghi nhật ký chi tiết vào server log console.error. Tại môi trường production (NODE_ENV=production), thông báo trả về được che giấu là "Lỗi hệ thống, vui lòng thử lại sau"; không gửi stack trace, câu lệnh SQL nội bộ, connection string hay biến môi trường cho client. Trường stack chỉ được mở khi NODE_ENV=development cục bộ.
- **Fallback 404 handler:** Đặt sau toàn bộ routes và trước errorHandler trong src/server.ts để xử lý các đường dẫn không tồn tại.

6. Triển Khai Phần Mở Rộng Buổi 10 (Thống Kê, Stored Procedure & JOIN) (Slide 20)

Hệ thống đã triển khai 4 kỹ thuật truy vấn nâng cao trên CSDL SQL Server thật:

- **A. Raw Query:** Sử dụng sequelize.query("SELECT maLop, COUNT(*) AS totalStudents FROM dbo.SinhVien GROUP BY maLop ORDER BY maLop ASC;", { type: QueryTypes.SELECT }) để báo cáo số lượng sinh viên theo lớp.
- **B. Sequelize Method:** Thống kê bằng hàm ORM: SinhVien.findAll({ attributes: ["maLop", [fn("COUNT", col("maSV")), "totalStudents"]], group: ["maLop"], order: [["maLop", "ASC"]] }).
- **C. Stored Procedure:** Tạo thủ tục chỉ đọc dbo.sp_ThongKeSinhVienTheoLop trong SQL Server và thực thi qua lệnh EXEC dbo.sp_ThongKeSinhVienTheoLop với quyền EXECUTE an toàn cho user sv_app.
- **D. JOIN Quan Hệ 1-N:** Sử dụng SinhVien.findAll({ include: [{ model: Lop, as: "lop", attributes: ["tenLop"] }], order: [["maSV", "ASC"]] }) lấy sinh viên kèm tên lớp học thực tế.

7. Kỹ Năng AI-Driven & Phản Biện Mã Nguồn Bắt Buộc (Slide 21)

Quá trình thực hành áp dụng quy trình CTCO 5 bước (Context - Task - Constraint - Output) kết hợp kiểm tra độc lập trên môi trường SQL Server thật:

Hai tình huống phản biện mã nguồn AI đề xuất sai:

1. Phản biện phân trang giả bằng JavaScript: ***Tình huống:*** AI đề xuất gọi SinhVien.findAll() nạp toàn bộ bảng về bộ nhớ rồi dùng .slice(offset, offset + limit) để cắt trang. ***Phân tích kỹ thuật:*** Cách làm này gây quá tải bộ nhớ Node.js (Out-of-Memory) khi dữ liệu tăng lớn, làm mất toàn bộ ý nghĩa tối ưu hóa I/O của phân trang. Giải pháp sửa đổi: Bắt buộc dùng findAndCountAll với offset và limit trực tiếp trong câu lệnh SQL gửi đến Database.

2. Phản biện lộ stack trace trong môi trường Production: ***Tình huống:*** AI đề xuất trả err.stack và err.message nguyên bản cho mọi lỗi 500 gửi về client. ***Phân tích kỹ thuật:*** Lỗi này làm lộ cấu trúc thư mục nội bộ, phiên bản thư viện và logic máy chủ (Information Disclosure). Giải pháp sửa đổi: Cấu hình middleware kiểm tra NODE_ENV; khi ở chế độ production, thông báo trả về bắt buộc là "Lỗi hệ thống, vui lòng thử lại sau" và không chứa trường stack.

8. Danh Mục Hồ Sơ Bàn Giao (Slide 22)

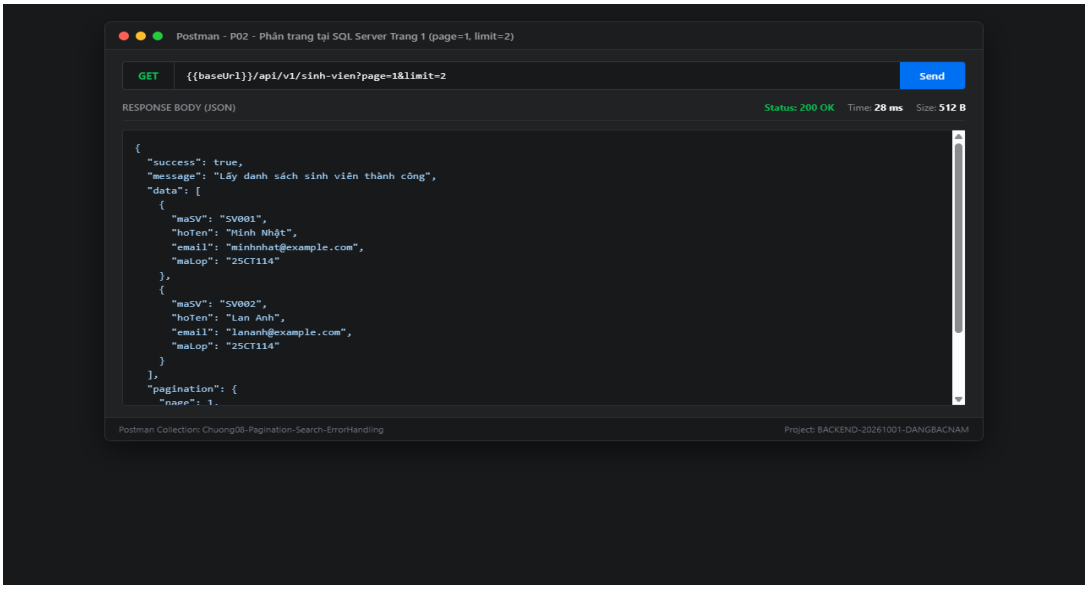
Toàn bộ hồ sơ thực hành Chương 08 được chuẩn bị và lưu trữ đầy đủ trong dự án theo đúng quy định:

Thành phần hồ sơ	Vị trí lưu trữ trong dự án	Mô tả nội dung kỹ thuật

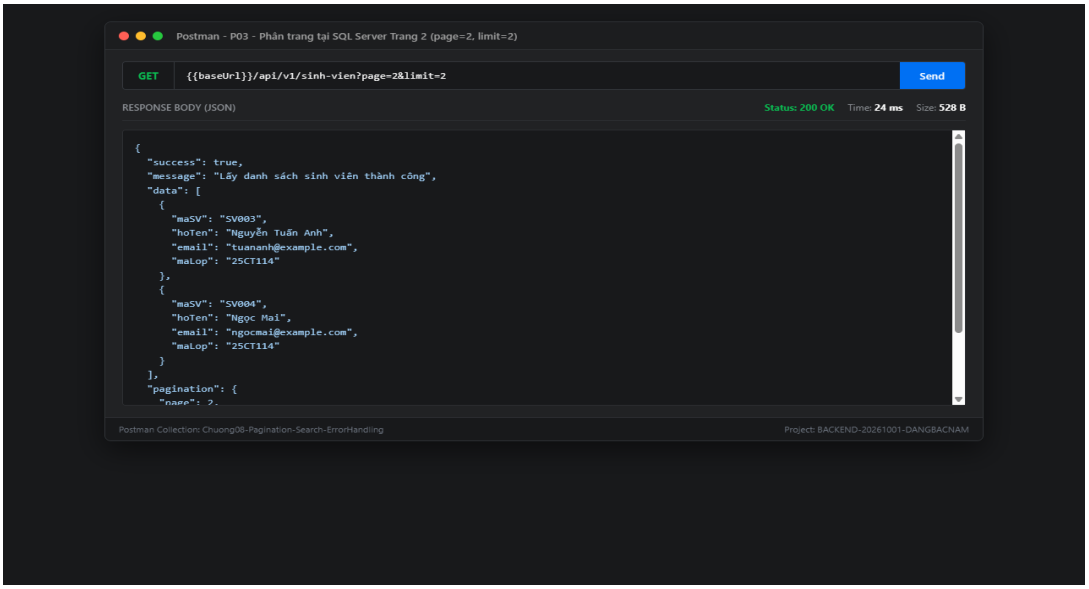
Mã nguồn Backend & Frontend	src/ và frontend/src/	TypeScript ESM, Sequelize 6, React Vite
Tài liệu OpenAPI 3.0	docs/openapi.yaml	Mô tả query params và phản hồi 200/400
Postman Collection JSON	docs/Chuong08-Pagination-Error.postman_collection.json	27 requests kiểm thử P01-P17 và E01-E10
Báo cáo Kết quả Kiểm thử	docs/test-results.md	Ma trận đối chiếu chi tiết các kịch bản
Hình ảnh minh chứng thực nghiệm	docs/screenshots/	11 ảnh chụp Postman, React UI, Swagger
Báo cáo PDF & Word chính thức	docs/Bao_Cao_Thuc_Hanh_Chuong_08.pdf (.docx)	Tài liệu báo cáo kỹ thuật hoàn chỉnh

9. Minh Chứng Thực Nghiệm Hình Ảnh

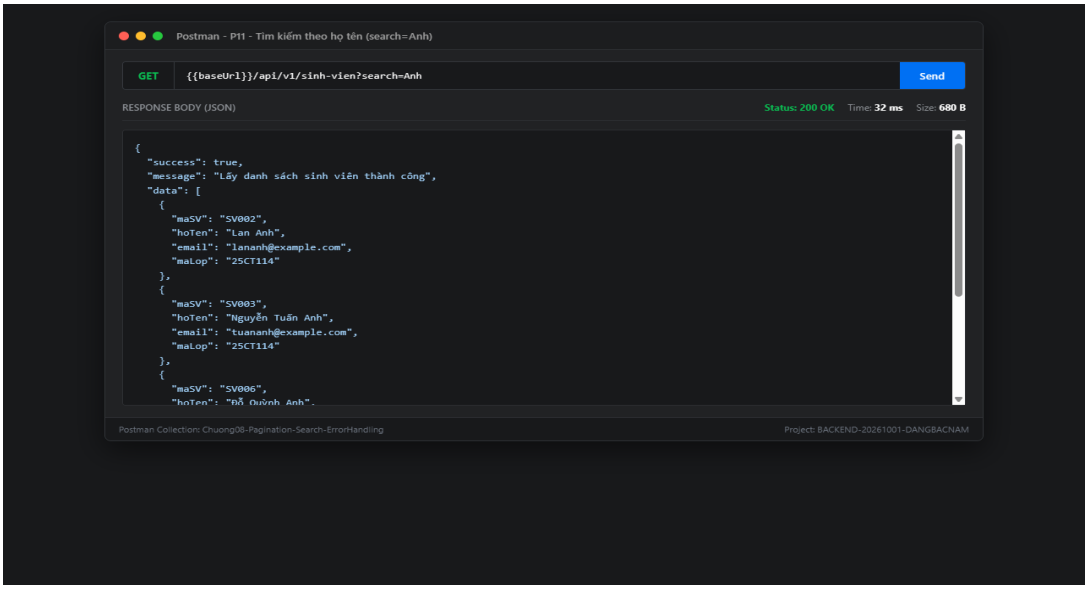
Dưới đây là hình ảnh chụp từ các công cụ kiểm thử Postman, giao diện React Vite và Swagger UI:



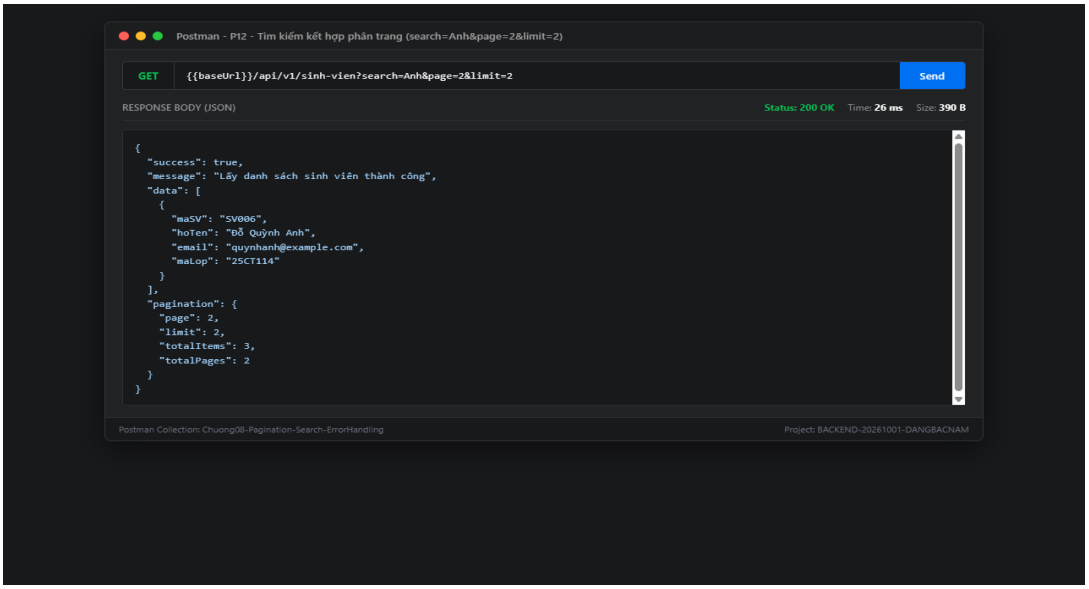
Hình minh chứng: Ca P02 — Phân trang tại SQL Server Trang 1 (page=1, limit=2: SV001, SV002, totalItems=7)



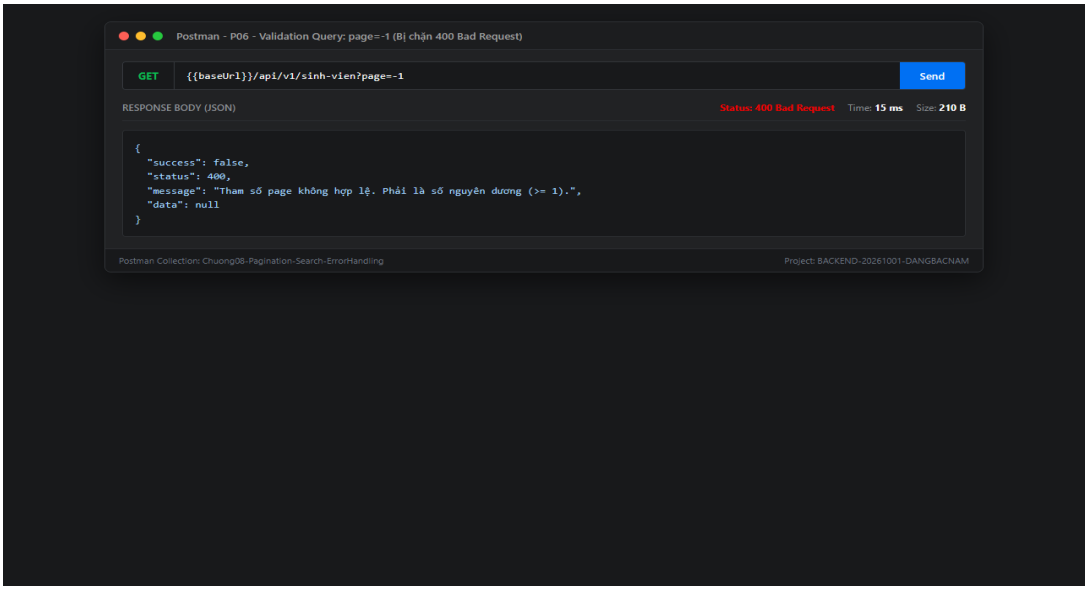
Hình minh chứng: Ca P03 — Phân trang tại SQL Server Trang 2 (page=2, limit=2: SV003, SV004, không trùng trang 1)



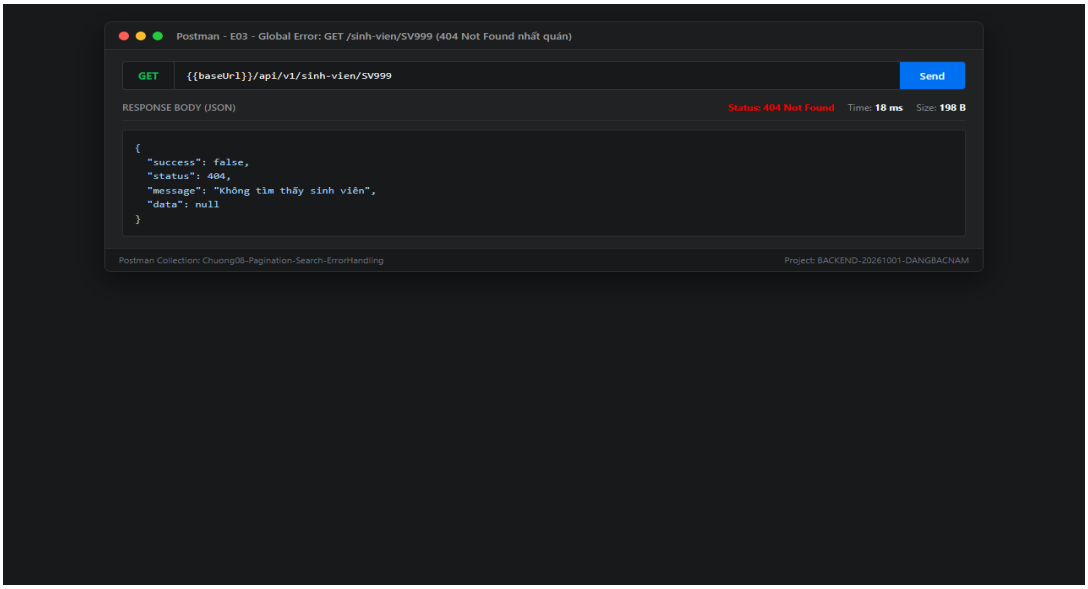
Hình minh chứng: Ca P11 — Tìm kiếm theo họ tên (search=Anh: lọc ra 3 sinh viên có tên chứa Anh)



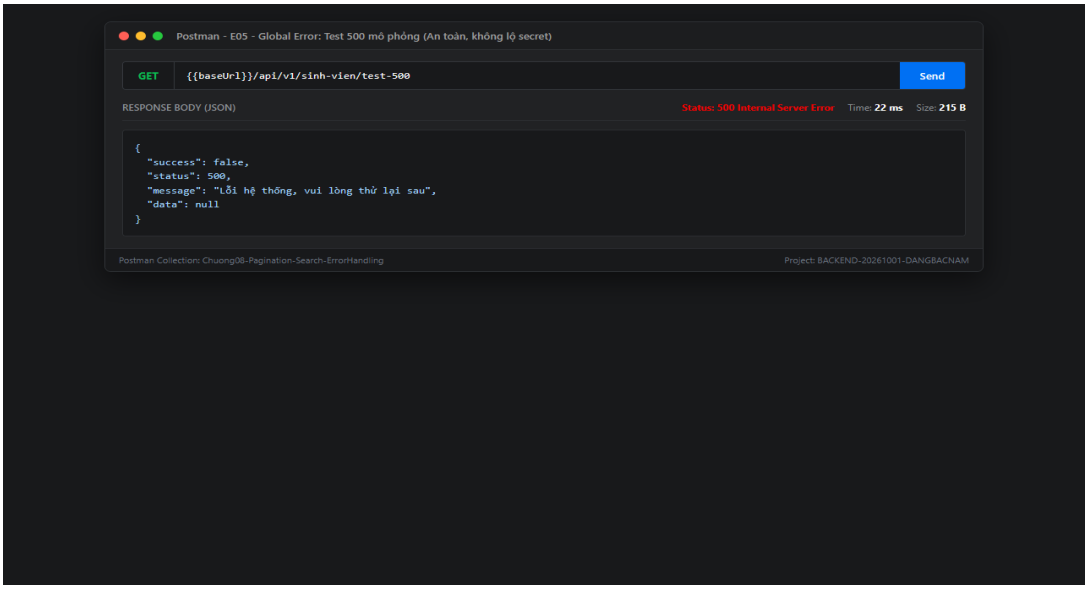
Hình minh chứng: Ca P12 — Tìm kiếm kết hợp phân trang (search=Anh&page=2&limit=2: totalItems=3, lấy Đỗ Quỳnh Anh)



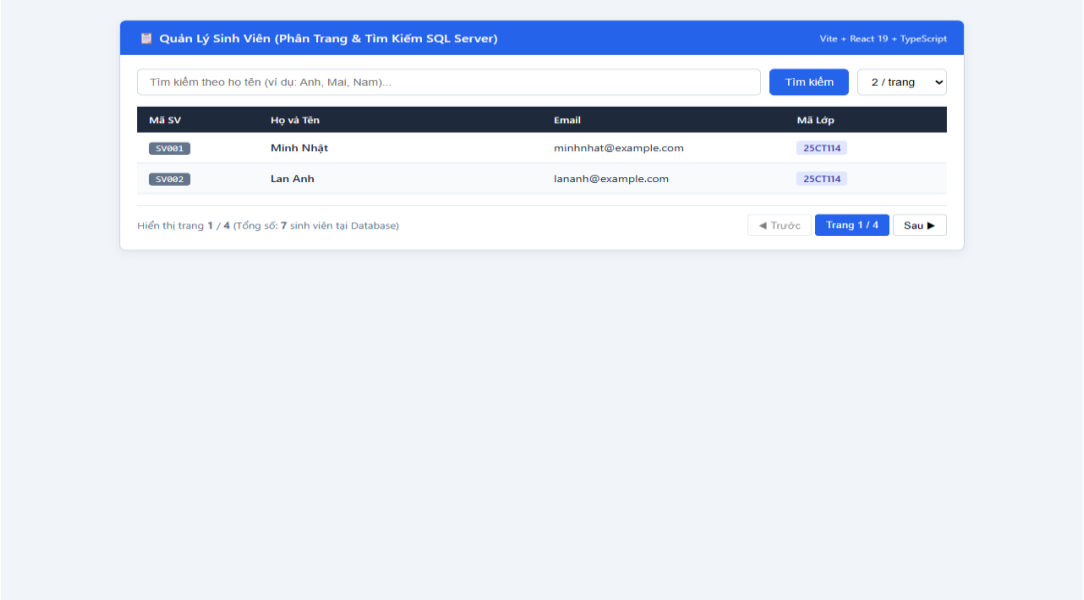
Hình minh chứng: Ca P06 — Chặn 400 Bad Request khi truyền page=-1 không hợp lệ



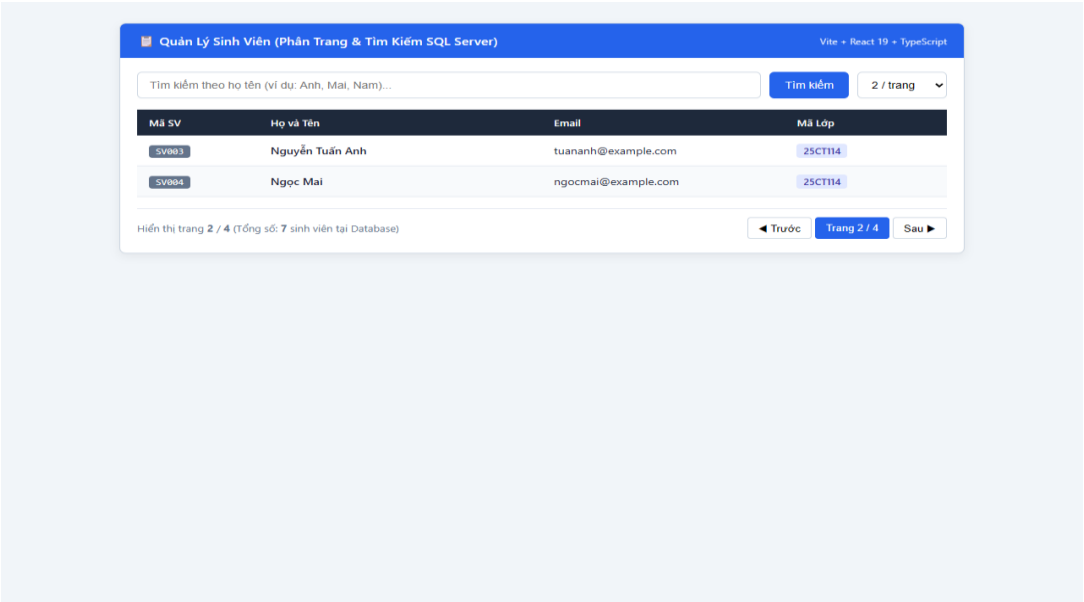
Hình minh chứng: Ca E03 — Global Error Handling: GET /sinh-vien/SV999 trả 404 nhất quán, data=null



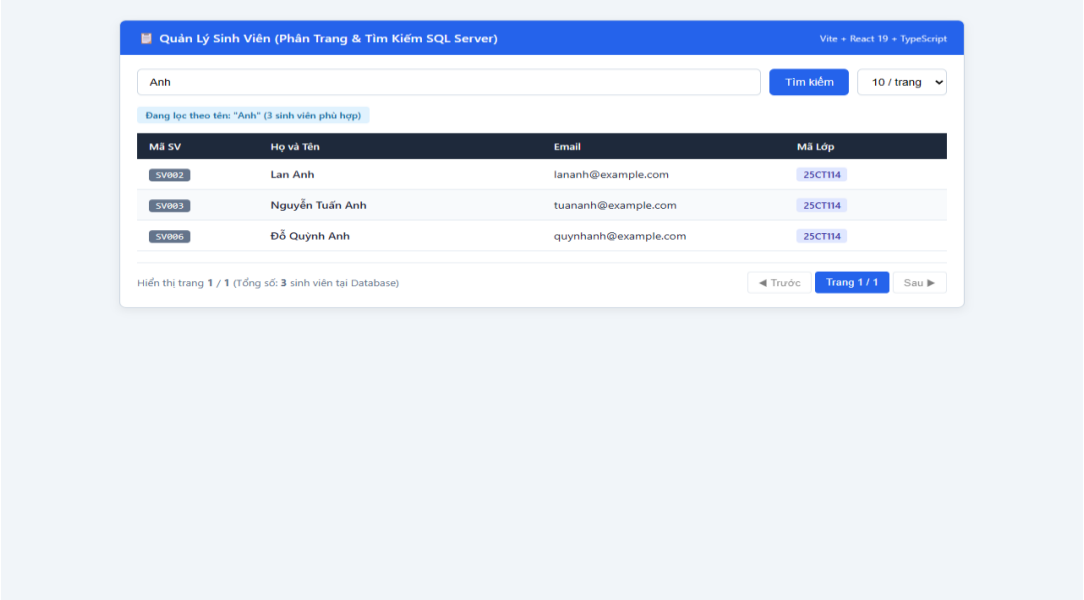
Hình minh chứng: Ca E05 — Global Error Handling: Lỗi 500 mô phỏng an toàn, không lộ secret



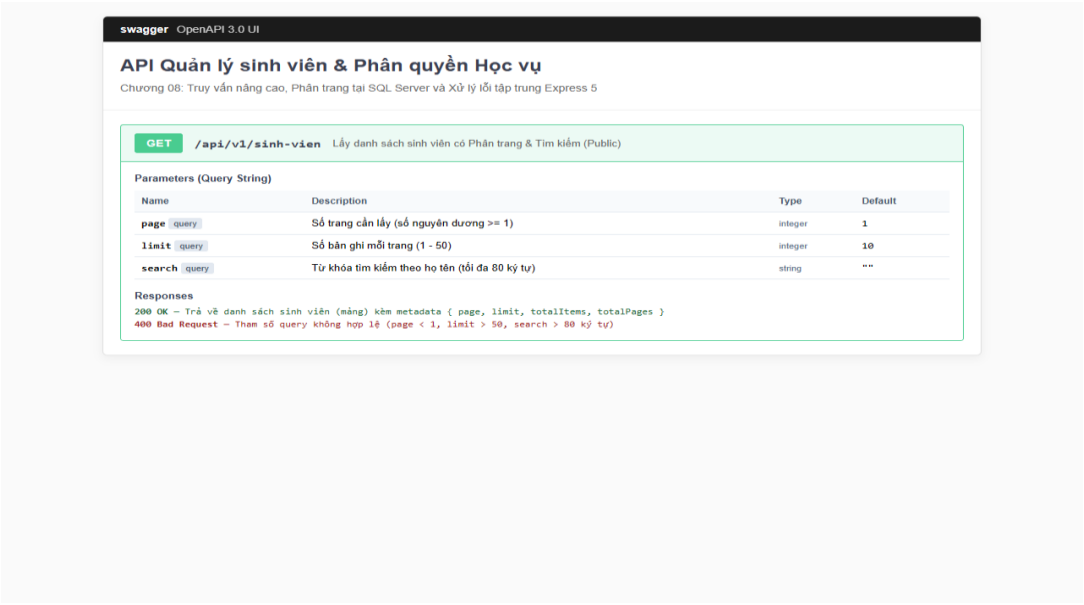
Hình minh chứng: Giao diện React Vite hiển thị Trang 1 (2 sinh viên/trang, nút Trước bị disabled, tổng 7 sinh viên)



Hình minh chứng: Giao diện React Vite hiển thị Trang 2 (phát sinh request GET page=2, dữ liệu chuyển đổi trên UI)



Hình minh chứng: Giao diện React Vite tìm kiếm "Anh" (bảng lọc 3 sinh viên, badge thông báo từ khóa)



Hình minh chứng: Giao diện Swagger UI (/api-docs) mô tả đầy đủ query parameters page, limit, search và response 200/400

10. Bảng Kịch Bản Kiểm Thử & Hiện Trạng Phản Hồi (Slide 13, 14, 18)

Bảng 1: Kịch bản kiểm thử Phân trang & Tìm kiếm tại Database (P01 đến P17)

Mã Ca	Tình huống kiểm thử	HTTP Kỳ vọng	HTTP Thực tế	Hiện trạng phản hồi
P01	GET /api/v1/sinh-vien mặc định	200	**200**	data là mảng, metadata page=1, limit=10

P02	GET ?page=1&limit=2 (Trang 1)	200	**200**	2 bản ghi đầu theo maSV ASC (SV001, SV002)
P03	GET ?page=2&limit=2 (Trang 2)	200	**200**	2 bản ghi tiếp theo (SV003, SV004)
P04	GET ?page=3&limit=2 (Trang 3)	200	**200**	Lấy các bản ghi tiếp (SV005, SV006)
P05	GET ?page=999&limit=2 (Vượt tổng trang)	200	**200**	data=[], totalItems=7 chính xác
P06	Validation: GET ?page=-1 (Số âm)	400	**400**	Chặn 400: page phải là số nguyên dương
P07	Validation: GET ?page=0 (Bằng 0)	400	**400**	Chặn 400: Trang bắt đầu từ 1
P08	Validation: GET ?page=1.5 (Thập phân)	400	**400**	Chặn 400: Không chấp nhận thập phân
P09	Validation: GET ?limit=abc (Chữ cái)	400	**400**	Chặn 400: limit phải là số nguyên
P10	Validation: GET ?limit=51 (Vượt 50)	400	**400**	Chặn 400: Vượt giới hạn cho phép 1-50
P11	GET ?search=Anh (Tìm theo họ tên)	200	**200**	Lọc đúng 3 sinh viên có tên chứa Anh
P12	GET ?search=Anh&page=2&limit=2	200	**200**	totalItems=3 (lọc), trang 2 lấy Đỗ Quỳnh Anh
P13	GET ?search=khongco (Không kết quả)	200	**200**	data=[], totalItems=0, totalPages=0
P14	GET ?search= Anh (Có khoảng trắng)	200	**200**	Tự động trim thành Anh, trả kết quả đúng
P15	Validation: GET ?search=[81 ký tự]	400	**400**	Chặn 400: Từ khóa vượt quá 80 ký tự

P16	Kiểm chứng SQL OFFSET/FETCH qua log	200	**200**	SQL Server áp dụng OFFSET 2 FETCH NEXT 2
P17	Hồi quy CRUD: GET /sinh-vien/SV001	200	**200**	Tìm thấy SV001 - Minh Nhật không đổi

Bảng 2: Kịch bản kiểm thử Xử lý lỗi toàn cục & Hồi quy bảo mật (E01 đến E10)

Mã Ca	Tình huống kiểm thử	HTTP Kỳ vọng	HTTP Thực tế	Hiện trạng phản hồi
E01	GET ?page=-5 (Lỗi tham số âm)	400	**400**	JSON chuẩn: success=false, status=400, data=null
E02	GET ?limit=abc (Lỗi kiểu số)	400	**400**	Chặn trước tầng DB, không phát sinh lỗi SQL
E03	GET /sinh-vien/SV999 (Không tìm thấy)	404	**404**	404 Not Found nhất quán: status=404, data=null
E04	GET /duong-dan-khong-co (Fallback)	404	**404**	Fallback 404 router hoạt động đúng chuẩn
E05	Lỗi 500 mô phỏng qua route test-500	500	**500**	Không lộ SQL/secret, status=500, data=null
E06	NODE_ENV=production, kiểm tra 500	500	**500**	Không có trường stack, message bảo mật
E07	NODE_ENV=development, kiểm tra 500	500	**500**	Trường stack chỉ hiển thị tại dev máy cục bộ
E08	Hồi quy Auth: GET /profile thiếu token	401	**401**	Chặn 401 Unauthorized do thiếu Bearer Token
E09	Hồi quy RBAC: Student ép DELETE	403	**403**	Chặn 403 Forbidden tài khoản student, DB an toàn
E10	Hồi quy Multer: Upload sai field photo	400	**400**	Báo lỗi 400 yêu cầu đúng tên field "avatar"

Bảng 3: Kịch bản kiểm thử Truy vấn Mở rộng Buổi 10 (X01 đến X04)

Mã Ca	Phương pháp kiểm thử	Tuyến đường API / Lệnh thực thi	Hiện trạng phản hồi
X01	Raw SQL Query	GET /api/v1/sinh-vien/thong-ke?method=raw	Trả về mảng thống kê số lượng theo lớp
X02	Sequelize Group & Count	GET /api/v1/sinh-vien/thong-ke?method=sequelize	Gom nhóm maLop và đếm totalStudents
X03	Stored Procedure	GET /api/v1/sinh-vien/thong-ke?method=sp	Thực thi dbo.sp_ThongKeSinhVienTheoLop
X04	Eager Loading JOIN Model	GET /api/v1/sinh-vien/kem-lop	Trả về sinh viên kèm đối tượng Lop ({ tenLop })

11. Trả Lời Chi Tiết 12 Câu Hỏi Bảo Vệ & Bài Tập Mở Rộng (Slide 23)

Câu 01. page, limit và offset khác nhau thế nào? page=5 và limit=20 thì offset là bao nhiêu? **Trả lời:** - page: Chỉ số trang hiện tại mà client muốn lấy (bắt đầu từ 1). - limit: Số lượng bản ghi tối đa được phép hiển thị trên mỗi trang. - offset: Số lượng bản ghi mà hệ quản trị CSDL cần bỏ qua từ đầu tập kết quả trước khi bắt đầu lấy dữ liệu cho trang hiện tại. Công thức tính: offset = (page - 1) * limit. Với page = 5 và limit = 20: offset = (5 - 1) * 20 = 80.

Câu 02. Vì sao phải ORDER BY maSV trước khi dùng OFFSET/FETCH trong SQL Server? **Trả lời:** Trong SQL Server và chuẩn ANSI SQL, mệnh đề OFFSET/FETCH bắt buộc phải đi kèm mệnh đề ORDER BY. Nếu không có ORDER BY, thứ tự bản ghi trả về từ CSDL là không tất định (non-deterministic). Khi đó, các lần gọi trang 1 và trang 2 có thể trả về các bản ghi trùng lặp hoặc bỏ sót dữ liệu. Khóa maSV là khóa chính duy nhất, đảm bảo thứ tự dữ liệu cố định và nhất quán giữa các trang.

Câu 03. findAll() khác findAndCountAll() ra sao? rows và count dùng để làm gì? **Trả lời:** - findAll(): Chỉ thực thi một truy vấn SELECT đơn thuần để lấy danh sách bản ghi thỏa điều kiện where kèm limit/offset. - findAndCountAll(): Thực thi đồng thời việc đếm tổng số bản ghi thỏa điều kiện where (trước khi phân trang) và lấy mảng bản ghi của trang hiện tại. - count: Là tổng số dòng dữ liệu thỏa điều kiện lọc trên toàn bộ bảng, dùng để tính tổng số trang (totalPages). - rows: Là mảng các bản ghi cụ thể thuộc trang được yêu cầu, gửi về cho client hiển thị.

Câu 04. Vì sao totalPages = Math.ceil(totalItems / limit)? Khi không có dữ liệu thì trả gì? **Trả lời:** Hàm Math.ceil() làm tròn lên số nguyên gần nhất. Ví dụ với 5 sinh viên và limit = 2: 5 / 2 = 2.5, làm tròn lên là 3 trang (trang 1 có 2, trang 2 có 2, trang 3 có 1). Nếu làm tròn xuống (Math.floor), trang cuối cùng sẽ bị mất. Khi không có dữ liệu (totalItems = 0), hệ thống trả về data: [], totalPages: 0; thuộc tính page vẫn phản ánh giá trị client yêu cầu.

Câu 05. Vì sao lọc search theo hoTen phải xảy ra trước khi tính count và lấy trang? Trả lời: Mệnh đề WHERE lọc theo hoTen phải được áp dụng trên toàn bộ bảng để xác định chính xác tập bản ghi phù hợp. Nếu phân trang trước rồi mới lọc trên kết quả trang đó, tổng số bản ghi (count) và tổng số trang sẽ sai lệch, đồng thời bỏ sót các bản ghi nằm ở các trang khác (ví dụ sinh viên Lan Anh ở vị trí thứ 30 sẽ bị mất nếu chỉ lấy trang 1 gồm 10 bản ghi đầu rồi mới lọc).

Câu 06. Nếu client gửi page="abc" hoặc limit=-2, lỗi phù hợp là 400 hay 500? Vì sao? Trả lời: Mã lỗi phù hợp là 400 Bad Request. Đây là lỗi do dữ liệu đầu vào từ phía người dùng gửi lên không hợp lệ, không tuân thủ quy ước giao tiếp API. Trả về mã 500 (Internal Server Error) là sai ngữ nghĩa HTTP vì server không bị lỗi nội bộ hoặc crash, và lỗi cần được chặn lại ngay tại tầng validation trước khi truyền xuống CSDL.

Câu 07. Vì sao không nên trả data={rows,pagination} thay cho data[] của chương trước? Trả lời: Việc thay đổi kiểu dữ liệu của trường data từ mảng [] thành đối tượng {} sẽ làm vỡ giao diện Frontend của Chương 07 và các client cũ đang đọc res.data.data.map(...). Giữ nguyên data là mảng và đặt pagination là thuộc tính độc lập ở cấp cao nhất bảo đảm tính tương thích ngược (Backward Compatibility).

Câu 08. Axios gửi SQL tới Database hay gửi HTTP tới Backend? Chứng minh bằng Network. Trả lời: Axios chạy trên trình duyệt người dùng chỉ gửi HTTP GET Request mang query parameters (ví dụ /api/v1/sinh-vien?page=2&limit=2) tới máy chủ Express Backend. Trình duyệt hoàn toàn không có kết nối TCP tới SQL Server (cổng 1433/14333) và không gửi câu lệnh SQL. Tab Network của DevTools chứng minh rõ ràng đây là HTTP Request với URL, Query String và JSON Response.

Câu 09. AppError và Global Error Middleware khác nhau về trách nhiệm? Trả lời: - AppError: Là lớp biểu diễn lỗi nghiệp vụ dự kiến (Operational Error), mang theo mã trạng thái HTTP (statusCode) và thông điệp lỗi (message), được throw từ Controller, Service hoặc Validator. - Global Error Middleware: Là hàm đón đầu ở cuối pipeline Express (4 tham số), có trách nhiệm phân loại lỗi, ghi log hệ thống, che giấu chi tiết nhạy cảm và đóng gói cấu trúc JSON phản hồi duy nhất về cho Client.

Câu 10. Express 5 chuyển lỗi từ async handler tới middleware như thế nào? Trả lời: Trong Express 5, khi một async route handler trả về một Promise bị Reject (hoặc throw lỗi bên trong hàm async), Express 5 sẽ tự động bắt lấy ngoại lệ đó và chuyển tiếp tới Error Middleware tiếp theo, tương đương với việc gọi next(err). Lập trình viên không cần phải bọc try-catch lặp đi lặp lại hay sử dụng thư viện ngoài như express-async-errors.

Câu 11. Vì sao không được hiển thị stack trace và connection string ở production? Trả lời: Hiển thị stack trace và connection string làm lộ cấu trúc thư mục máy chủ, tên tài khoản, mật khẩu CSDL, phiên bản thư viện và logic thuật toán nội bộ. Kẻ tấn công có thể khai thác các thông tin này để thực hiện tấn công leo thang đặc quyền hoặc chiếm quyền kiểm soát máy chủ. Do đó, production bắt buộc phải che giấu bằng thông điệp chung "Lỗi hệ thống, vui lòng thử lại sau".

Câu 12. Nêu ba bằng chứng để chứng minh nâng cấp Chương 08 không làm hỏng Chương 07. Trả lời: 1. Kịch bản kiểm thử Auth & RBAC (Chương 06): Script test:24 xác nhận các route bảo vệ hoạt động đúng như thiết kế. 2. Kịch bản kiểm thử Upload Avatar

(Chương 07): Script test:upload xác nhận luồng upload và phục vụ file tĩnh /uploads/<uuid>.<ext> qua HTTP 200 OK. 3. Tính tương thích giao diện React Vite: Giao diện vừa hỗ trợ tìm kiếm và phân trang mới, vừa giữ nguyên Profile Card hiển thị ảnh đại diện và chức năng Xóa của quyền Admin.

Giải bài tập mở rộng (Slide 23):

• **So sánh COUNT theo maLop bằng Raw Query và Sequelize group:** Cả hai phương pháp đều sinh ra câu lệnh SQL có mệnh đề GROUP BY maLop và hàm tổng hợp COUNT(maSV). Raw Query cho phép lập trình viên kiểm soát trực tiếp câu lệnh T-SQL tối ưu, trong khi Sequelize group cung cấp tính trừu tượng hóa cao, an toàn kiểu dữ liệu và tránh được các lỗi cú pháp thủ công. • **JOIN để lấy tenLop từ bảng Lop:** Sử dụng SinhVien.findAll({ include: [{ model: Lop, as: "lop", attributes: ["tenLop"] }] }). Quan hệ được thiết lập dựa trên điều kiện khóa ngoại SinhVien.maLop = Lop.maLop, cho phép lấy thông tin sinh viên kèm tên lớp tương ứng trong một truy vấn duy nhất. • **Về việc lấy 5 sinh viên mới nhất theo cột ngày tạo:** Bảng dbo.SinhVien hiện tại không có cột createdAt hoặc ngày tạo bản ghi. Trong kiến trúc CSDL quan hệ chuẩn, không được tự suy đoán hoặc giả định sự tồn tại của cột thời gian. Do đó, nếu cần lấy 5 bản ghi mới nhất, bắt buộc phải có script bổ sung cột thời gian thực tế vào bảng trước khi thực hiện ORDER BY createdAt DESC. • **Đánh giá Stored Procedure so với Sequelize:** Stored Procedure được biên dịch và lưu kế hoạch thực thi (Execution Plan) sẵn tại SQL Server, giúp tối ưu hiệu năng cho các tác vụ tổng hợp dữ liệu quy mô lớn và cho phép phân quyền chặt chẽ (chỉ cấp EXECUTE mà không cấp quyền đọc bảng). Tuy nhiên, nhược điểm là khó quản lý phiên bản trong Git và phụ thuộc vào hệ quản trị CSDL cụ thể.

12. Hướng Dẫn Vận Hành Hệ Thống

Bảng tổng hợp các lệnh kiểm thử và vận hành hệ thống trong suốt quá trình triển khai:

Bước thực hiện	Lệnh thực thi dòng lệnh (Command)	Mục đích & Đầu ra kiểm tra
1. Khởi động Backend	pnpm run dev	Chạy tại http://localhost:5000
2. Khởi động Frontend	pnpm run dev:frontend	Mở tại http://localhost:5173
3. Kiểm thử Chương 08	pnpm run test:c8	Kiểm thử 25 ca P01-P17 và E01-E10
4. Kiểm thử mở rộng Buổi 10	npx tsx src/scripts/test-extensions.ts	Raw Query, SP, Sequelize, JOIN

5. Biên dịch mã nguồn	pnpm build && pnpm build:frontend	Kiểm tra biên dịch TypeScript

Báo cáo được hoàn thiện và xác nhận lưu trữ tại repository.